

Architecture et Programmation Web

Séance 1 : Introduction au Web

Nicolas Burny

6 mars 2026

ECAM

Objectifs de la séance

1. Comprendre l'évolution du Web (1.0 → 3.0).
2. Décortiquer une requête HTTP (URL, Headers, Body).
3. **Refresher Python** : Connaitre les bases du langage.
4. **Hello World** : Créer votre première API avec Flask.

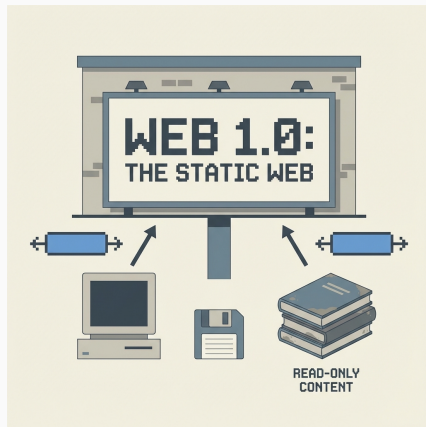
1. Histoire et Évolution

Ne confondez pas le réseau et l'application !

- **Internet (Infrastructure)** : Câbles, 4G, Routeurs, IP.
- **Web (Application)** : Sites, APIs, Navigateurs.

"Read-Only Web"

- **Utilisateur** : Consommateur passif.
- **Techno** : HTML, CSS basique, GIFs.
- **Business** : Sites vitrines, annuaires.
- **Caractéristique** : Pages figées, mise à jour manuelle des fichiers.



- **Heaven's Gate** : <https://www.heavensgate.com/>
- **Space Jam (1996)** : <https://www.spacejam.com/1996/index2.html>
- **Hiking.com.hk** : <http://www.hiking.com.hk/>
- **Lasse V** : <https://jkhub.org/sites/lassev/index.html>

Note : Ces sites sont toujours en ligne dans leur état d'origine !

"Read-Write Web"

- **Utilisateur** : Producteur de contenu (UGC).
- **Techno** : AJAX, Bases de données (SQL), JavaScript.
- **Business** : Facebook, Amazon, YouTube.
- **Caractéristique** : Interaction, commentaires, e-commerce dynamique.



Web 3.0 (2010+) : Le Web Intelligent

Un web plus intelligent, décentralisé et immersif.

- **Personnalisation IA** : Contenus adaptés aux comportements et préférences des utilisateurs.
- **Décentralisation** : La blockchain redonne le contrôle des données aux utilisateurs.
- **Immersion** : Réalité augmentée (AR) et virtuelle (VR) dans le commerce et la communication.
- **Éthique** : Réglementations, transparence et responsabilité numérique.



Web sémantique Les données sont organisées pour que les machines comprennent leur contexte → recherches plus précises.

Blockchain Données distribuées, non contrôlées par des entités centrales → confidentialité et sécurité.

IA & Machine Learning Prédiction des comportements, personnalisation en temps réel.

Internet des Objets Objets connectés intégrés au web → interaction fluide entre mondes physique et numérique.

Selon l'audience (Perspective Business)

- **APIs Privées (Interne)** : Utilisées uniquement au sein de l'entreprise pour connecter des systèmes (ex : RH vers Paie).
- **APIs de Partenariat** : Partagées avec des partenaires business spécifiques (ex : Uber intégrant Google Maps).
- **APIs Publiques (Open API)** : Accessibles à tous les développeurs externes (ex : API Twitter, Meteo).

Styles Architecturaux (Perspective Technique)

- **REST** : Le standard actuel du Web (simple, basé sur HTTP).
- **GraphQL** : Permet au client de demander exactement ce qu'il veut.
- **gRPC** : Protocole de communication plus performant.
- **WebSocket** : Communication en temps réel.
- **SOAP** : Protocole de communication plus ancien.

Le Web en action

Clients et Serveurs communiquent
via des Requêtes et des Réponses.

2. Anatomie d'une Requête

1. Vous tapez une URL.
2. **DNS** : Le navigateur convertit le nom de domaine en adresse IP.
3. Le navigateur envoie une **Requête**.
4. Le serveur traite la requête et renvoie une **Réponse**.
5. **Rendu** : Le navigateur interprète la réponse et affiche la page.

`https://www.google.com/search?q=flask`

- **Protocole** : `https` (HTTP sécurisée).
- **Hôte** : `www.google.com` (Serveur).
- **Chemin (Path)** : `/search` (Ressource).
- **Query Params** : `?q=flask` (Filtres).

Ce que le client envoie au serveur.

Méthode GET (Lire) ou POST (Écrire).

Headers Métadonnées (Langue, Navigateur).

Body Contenu utile (Données de formulaire).

Ce que le serveur envoie au client.

Status 200 OK, 404 Not Found, etc.

Headers Type de contenu (text/html, application/json, etc.).

Body Le contenu (HTML, JSON, etc.).

3. Python Refresher

Variables & Types (1/2)

Pas besoin de déclarer le type (Typage dynamique).

```
prix = 10.5           # Float  
nom = "ECAM"         # String  
actif = True          # Bool
```

La méthode moderne pour formater du texte.

```
nom = "Alice"  
age = 24  
print(f"User: {nom} ({age} ans)")  
# "User: Alice (24 ans)"
```

```
role = "Admin"
if role == "Admin":
    print("Accès complet")
elif role == "User":
    print("Lecture seule")
else:
    print("Accès refusé")
```

Boucle FOR : Quand on connaît le nombre d'itérations.

```
for i in range(3):  
    print(f"Tentative {i}")  
# 0, 1, 2
```

Boucle **WHILE** : Tant qu'une condition est vraie.

```
batterie = 10
while batterie > 0:
    print("En marche...")
    batterie -= 1
```

Listes (Lists)

Collection ordonnée et modifiable.

```
fruits = ["Pomme", "Poire"]  
fruits.append("Banane")  
print(fruits[0]) # Pomme
```

Dictionnaires (Dicts)

Structure de donnée au format clé-valeur.

```
produit = {  
    "nom": "Laptop",  
    "prix": 999,  
    "stock": True  
}  
  
print(produit["prix"]) # 999
```

Permet de réutiliser du code.

```
def tva(prix, taux=0.21):  
    return prix * taux
```

```
taxe = tva(100) # 21.0
```

Les Décorateurs

Permettent d'envelopper une fonction pour modifier son comportement sans toucher à son code.

```
def mon_decorateur(f):  
    def wrapper():  
        print("Avant l'exécution...")  
        f()  
        print("Après l'exécution.")  
    return wrapper
```

```
@mon_decorateur  
def dire_bonjour():  
    print("Bonjour !")
```

4. Introduction à Flask

Qu'est-ce que Flask ?

- Un **Micro-framework** Python.
- Minimaliste : Pas de modules installés par défaut.
- Extensible : On ajoute ce dont on a besoin.
- Utilisé par de nombreuses entreprises et projets.

Dans votre terminal :

1. Créer l'environnement virtuel

```
python -m venv venv
```

2. Activer

```
.\venv\Scripts\activate
```

3. Installer

```
pip install flask
```

Hello World (Code)

Créez un fichier app.py :

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello, world!"

if __name__ == "__main__":
    app.run(debug=True)
```

Route = URL → Fonction Python

- Si je vais sur /, Flask exécute `home()`.
- La valeur de retour ("Hello, world!") est envoyée au navigateur.

1. Le web a évolué du contenu statique vers une communication dynamique basée principalement sur des API.
2. Les clients et serveurs communiquent via des requêtes et des réponses.
3. Il existe différents types d'API, chacun avec ses propres avantages et inconvénients.
4. Python (langage) et Flask (framework) permettent de créer des API.

Questions ?